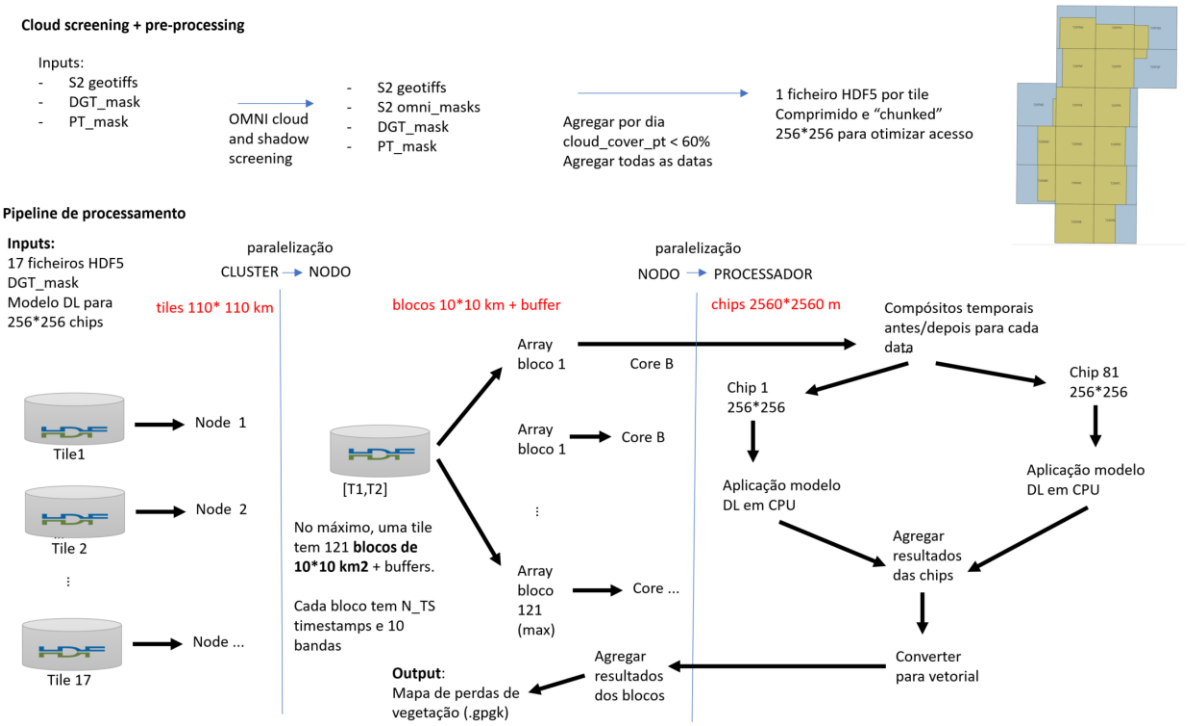
# S2CHANGE: Desenvolvimento de mapas de perdas recentes de floresta e mato em Portugal derivados de imagens de satélite

Contrato N.º 3044 de cooperação entre a Direção Geral do Território e o Instituto Superior de Agronomia

Entregável 4.4. Aplicação informática que possa ser integrada na cadeia de produção da DGT.

Resumo gráfico:




(O relatório é redigido em Inglês)

## Contents

Introduction ..... 2
Main parts of the pipeline ..... 2
Parallelization ..... 3
Potential vegetation loss dates ..... 3
Deep learning models ..... 4

# Introduction

The whole pipeline runs in the INCD HPC cluster. It is also available at https://github.com/S2change/vegetation_loss/tree/main/scripts/cpca070342024_shared/predict_pipeline. See the above graphical workflow that describes what blocks and chips are.

The main batch file **submit_slurm.sh** executes the following steps for each tile:

1. input_setup to get blocks and dates
2. distributes a block to each array:
    a. runs predict_block
    b. each array generates composites, shifted chips, predictions, votes, and returns results
3. aggregate_tile.py combines prediction results into full tile

Batch file **submit_tiles_batch.sh** submits the pipeline for many tiles in one command. It calls **submit_tile.sh** once per tile — each tile becomes its own independent SLURM array job + aggregator, so tiles run in parallel and SLURM spreads their block tasks across nodes. Per-tile load is still bounded by each submission's MAX_CONCURRENT cap, so submitting all tiles at once is safe (the scheduler queues whatever doesn't fit).

The only thing that differs between tiles is the HDF5 file; every other input is identical and forwarded verbatim to each submit_tile.sh call. Per tile the wrapper derives TILE_ID (the .h5 filename without extension) and OUTPUT_DIR=<BASE_OUTPUT_DIR>/<TILE_ID>.

# Main parts of the pipeline

At the tile level (**submit_tile.sh**), the main parts of the pipeline are the following.

1. input_setup/ runs at the full tile level to determine temporal clustering + change dates to use (input_setup/determine_clusters_of_dates.py) and block grid layout (input_setup/hdf5_reader.py)
2. Each block is submitted as an array job, 1 block per job, with a limit that only 8 blocks per tile can be processing at one time (run_block_slurm.sh activates predict_block.py)

3. Each CPU reads its assigned block from the HDF5 file (input_setup/hdf5_reader.py), creates the before/after composites (composite_shift_chips/composite.py), and then creates vertical, horizontal, and diagonally shifted chips from the original layout, for a total of 81 before/after chip pairs (composite_shift_chips/shift_chips.py)
4. For each chip pair, model runs and generates predictions for changes per pixel (models/<inputted_model>/predict.py)
5. At the chip level, the predictions are morphologically closed (postprocess/chip_records.py) but there is no minimum area dropped yet
6. The chip prediction results are joined so that each pixel in the 4x4 chip live area has 4 prediction results, which are counted as votes. Pixels with votes under the threshold are dropped (postprocess/vote.py)
7. Block result is written as an npz file (postprocess/voted_output.py) and as a gpkg (distribute/polygonize.py). These results are saved to <main_output_directory>/block_outputs/
8. Once all blocks have been run, run_block_errors_slurm.sh creates a text write up of any blocks that failed to run
9. Once all blocks have been successfully processed, run_aggregate_slurm.sh activates agggregate_tile.py. This script takes all the block outputs, merges overlapping results with same date and class prediction together, and returns a single gpkg file with all the prediction results, a tif for each change date with the class prediction results, and both an npz and a parquet file with the results in tabular form.
10. The gpkg file filters out patches that are below minimum patch size, default set to 0.5 ha.

# Parallelization

Sentinel-2 reflectance data is stored in 17 hdf5 files, one per tile. Each hdf5 file contains, in compressed format, all data for the tile, for all 10 bands and all available timestamps.

Processing is parallelized at two levels: tile and block. A block is a 10 km by 10 km array of values, with a surrounding buffer. Blocks are processed in the HPC cluster in parallel and independently.

For each tile, blocks are aggregated (see previous section) to obtain a single gpkg file per tile. Finally, all 17 gpkg files are combined into a single national vegetation loss vector map.

# Potential vegetation loss dates

Since Sentinel-2 available dates have an irregular temporal distribution due the irregularity of the orbits combined with cloud cover, the user needs to decide which potential dates of change are to be analyzed for each tile. In particular, it is expectable that more potential dates are analyzed in regions with lower cloud cover and therefore denser image time series.

Script input_setup/determine_clusters_of_dates.py reads the available dates from the hdf5 file fr each tile. Then it computes the set of dates, based on parameters MAX_THETA and

MAX_CLUSTER_AMPLITUDE that control those choices. Figure 1 and Figure 2 illustrate two different choices of parameters.

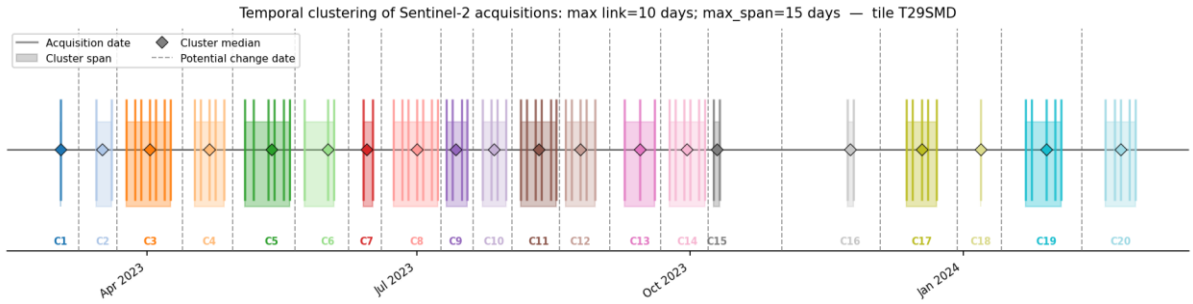



*Figure 1: Temporal aggregation of dates separated by at most 10 days, in temporal clusters that span at most 15 days (parameters MAX_THETA=10; MAX_CLUSTER_AMPLITUDE=15). Colors represent different clusters of dates. Vertical colored lines represent daily Sentinel-2 observations. Dotted black vertical lines indicate potential vegetation loss dates to be analyzed.*

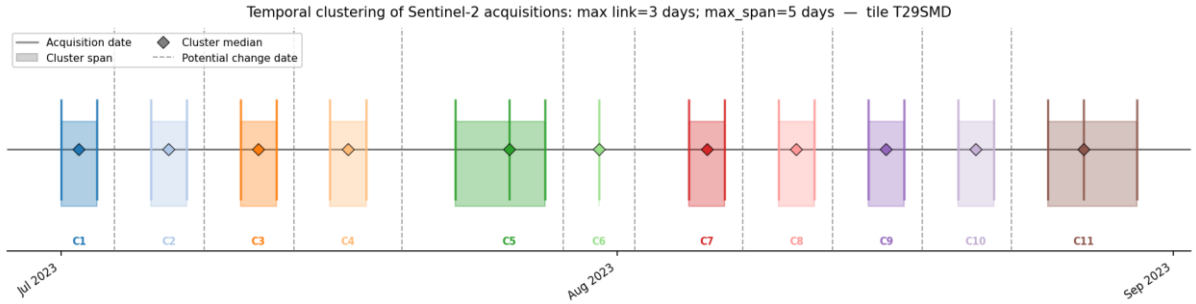



*Figure 2: Temporal aggregation of dates separated by at most 3 days, in temporal clusters that span at most 5 days (parameters MAX_THETA=3; MAX_CLUSTER_AMPLITUDE=5). Colors represent different clusters of dates. Vertical colored lines represent daily Sentinel-2 observations. Dotted black vertical lines indicate potential vegetation loss dates to be analyzed.*

# Deep learning models

Currently, three distinct deep learning models are available for prediction at the chip level using script models/<inputted_model>/predict.py.

To add new models or to update model's weights, the user just has to create or modify the model folder (see https://github.com/S2change/vegetation_loss/tree/main/scripts/cpca070342024_shared/predict_pipeline/models).

For instance, instructions to update model weights for enet_16bits are available at https://github.com/S2change/vegetation_loss/tree/main/scripts/cpca070342024_shared/predict_pipeline/models/enet_16bit/weights